

University of Jeddah  
College of Computer Science and Engineering  
Department of Software Engineering  
CCSW 325 – Software Construction

# LLM-Generated Variable Names vs. Human-Written Variable Names in Java: A Course Project Report

Joury Ghorab 2312080, Jana Akkad 2313200, Enas Alqarni 2314104

*Department of Software Engineering, University of Jeddah*

Instructor: Qamar Naith

**Abstract**—Variable naming is a foundational aspect of software quality that directly affects code readability and maintainability. With the rise of Large Language Models (LLMs) such as GPT-4, Gemini, and Llama, researchers have begun exploring whether these models can automate or assist with identifier naming tasks traditionally reliant on human judgment. This report presents a complete course project on this topic. It begins with a literature review of ten relevant academic studies on LLM-based identifier naming, automated variable renaming, and LLM-versus-human comparisons in software construction. It then describes a detailed project plan, followed by an empirical study in which a curated dataset of twenty-five Java code snippets was modified to remove meaningful identifiers, processed through ChatGPT (GPT-4) using a designed prompt, and evaluated using both automatic metrics (exact match and edit-distance-based similarity) and a human survey of eight student participants. Results indicate that LLM-generated variable names are highly readable and frequently preferred for clarity, although exact agreement with the original human-written names remains limited, confirming that LLMs are a strong assistive tool for naming rather than a complete replacement for developer judgment.

**Index Terms**—Large Language Models, Variable Naming, Identifier Naming, Java, Software Construction, Code Readability, ChatGPT, Automated Renaming.

## I. INTRODUCTION

Variable naming is widely recognized as a critical component of software quality. Poor or ambiguous names impede developer understanding and increase maintenance costs [2]. Studies indicate that identifiers account for approximately 70% of source code characters, making naming quality a primary driver of code comprehension [6].

The rapid advancement of Large Language Models (LLMs) such as GPT-4 [4], Gemini, and Llama [10] has opened a new avenue of research: can these models generate variable names that are as meaningful and convention-compliant as those written by experienced human developers?

This report surveys ten academic studies that collectively establish the theoretical and methodological foundation for a study comparing LLM-generated variable names against human-written variable names in Java code. The papers cover neural identifier naming, automated variable renaming, LLM-versus-human coding comparisons, code documentation evaluation, and prompt engineering. The report then presents a project plan and an empirical study that directly addresses the gap identified in the reviewed literature.

## II. LITERATURE REVIEW

### A. Wang et al. (2024) – *How Does Naming Affect LLMs on Code Analysis Tasks?*

**Idea:** This study investigates how identifier naming quality influences the performance of LLMs on code analysis tasks such as code summarization, clone detection, and defect prediction.

**Method:** The authors systematically obfuscated variable names in existing code corpora and measured the resulting drop in LLM performance across multiple tasks, experimenting with both GPT-based and BERT-based models.

**Dataset:** Multiple publicly available Java and Python code datasets.

**Evaluation Metrics:** BLEU score, accuracy, and F1 score across downstream tasks.

**Results:** Obfuscated or semantically poor variable names significantly degraded LLM performance, confirming that naming quality is critical not only for human comprehension but also for model reasoning.

**Reflection:** This paper establishes the core motivation of our study. If naming affects LLMs so strongly, then how LLMs generate names becomes even more important to investigate. A key limitation is that the study measures the effect of naming on LLMs rather than the ability of LLMs to generate good names. Future work should directly benchmark name-generation quality [1].

### B. Mastropaolo et al. (2023) – *Automated Variable Renaming: Are We There Yet?*

**Idea:** This large-scale empirical study explores data-driven approaches for automatically recommending meaningful variable names to support rename refactoring operations in Java.

**Method:** Three techniques were evaluated: a statistical n-gram language model and two deep learning models (T5 and CugLM), all trained and tested on Java repositories mined from GitHub.

**Dataset:** Over 1,425 open-source Java repositories from GitHub.

**Evaluation Metrics:** Exact match rate and normalized edit distance between suggested and human-written names.

**Results:** Deep learning models substantially outperformed statistical models, but even the best models fell short of perfectly matching human variable names in many cases.

**Reflection:** This study provides the most direct methodological parallel to our project, using Java and GitHub datasets that closely align with our approach. Its main limitation is that it predates modern LLMs (GPT-4, Llama 3), making a comparison with current models a natural and valuable extension [2].

### C. Bavishi et al. (2018) – *Context2Name: Inferring Natural Variable Names*

**Idea:** Context2Name proposes a neural model to infer natural variable names from the usage context of minified or obfuscated JavaScript code.

**Method:** A lightweight static analysis captures variable usage contexts, which are encoded by an auto-encoder and decoded by a recurrent neural network (RNN) to predict natural names.

**Dataset:** A large corpus of real-world JavaScript code.

**Evaluation Metrics:** Prediction accuracy, compared against JSNice and JS-Naughty.

**Results:** Context2Name successfully predicted 47.5% of all minified identifiers with a mean inference time of 2.9 milliseconds, matching or exceeding prior tools while being significantly faster.

**Reflection:** This is foundational work showing that machine learning can meaningfully infer variable names. Its limitation is that it targets JavaScript with relatively simple RNN architectures. Extending this methodology to Java with modern LLMs represents a strong research contribution [3].

#### ***D. Hou & Ji (2024) – Comparing LLMs and Human Programmers for Generating Code***

**Idea:** This study comprehensively evaluates seven LLMs on code generation tasks, comparing their performance to human programmers on competitive programming benchmarks.

**Method:** GPT-4, Gemini Ultra, Claude 2, and four other LLMs were tested on LeetCode and GeeksforGeeks problems using multiple prompt strategies.

**Dataset:** Hundreds of competitive programming challenges at varying difficulty levels.

**Evaluation Metrics:** Pass rate and performance percentile compared to human participants.

**Results:** GPT-4 with an optimal prompting strategy outperformed 85% of human participants in most contests and showed strong capabilities in code translation and error correction.

**Reflection:** While focused on code generation rather than naming specifically, this study establishes the LLM-versus-human comparison as a rigorous methodology. Its limitation is that it does not assess naming quality or readability of the generated code [4].

#### ***E. Yeadon et al. (2024) – Human, GPT-3.5, and GPT-4 in a University Coding Course***

**Idea:** This paper compares the coding submissions of human students against those generated by GPT-3.5 and GPT-4 in a university course, including a human-versus-AI detection component.

**Method:** Blinded human markers evaluated submissions without knowing authorship. Prompt engineering techniques were applied to both models.

**Dataset:** Student programming submissions and AI-generated equivalents, publicly available on Figshare.

**Evaluation Metrics:** Marker scores, Likert-scale AI/human classification, and statistical significance tests.

**Results:** Prompt engineering significantly improved GPT-4 scores. Markers correctly identified AI-generated code with approximately 85% accuracy under binary classification.

**Reflection:** The human-evaluation methodology directly informs the survey component of our project. A limitation is that it does not assess variable naming quality specifically, which our study will address [5].

#### ***F. Siddiq et al. (2023) – LLMs for Code Documentation Generation***

**Idea:** This study evaluates GPT-3.5, GPT-4, Bard, Llama2, and StarChat on their ability to generate code documentation at inline, function, and file levels.

**Method:** A checklist-based evaluation system assessed documentation along five dimensions: accuracy, completeness, relevance, understandability, and readability.

**Dataset:** Real-world code snippets across multiple programming languages.

**Evaluation Metrics:** Checklist scores across five quality dimensions and generation time.

**Results:** All LLMs except StarChat outperformed original human-written documentation. Closed-source models (GPT-4, Bard) outperformed open-source alternatives. File-level documentation was consistently the weakest output.

**Reflection:** The checklist-based evaluation approach is directly applicable to our survey design for variable naming. A limitation is that documentation generation differs from identifier naming in scope and granularity [6].

### ***G. Ma et al. (2023) – ChatGPT for Software Engineering Tasks***

**Idea:** This study evaluates ChatGPT across multiple Java software engineering tasks including method name suggestion, bug fixing, code summarization, and code generation.

**Method:** Manual experimentation with ten samples per task, with qualitative analysis of outputs.

**Dataset:** Manually constructed Java code snippets across tasks.

**Evaluation Metrics:** Qualitative correctness assessment and manual comparison against ground truth.

**Results:** ChatGPT showed promising results in method naming and code summarization but struggled with complex semantic understanding and occasionally hallucinated non-existent facts.

**Reflection:** This is one of the earliest studies to cover naming-related tasks for Java, making it highly relevant to our project. Its main limitation is the very small sample size (10 samples per task), which our study addresses with a larger dataset [7].

### ***H. Cordeiro et al. (2024) – LLM Code Refactoring Capability***

**Idea:** This paper empirically evaluates ChatGPT’s ability to perform code refactoring on Java files, including readability, maintainability, and naming consistency.

**Method:** 40 Java files were submitted to ChatGPT with a structured refactoring prompt, and outputs were analyzed across multiple quality attributes.

**Dataset:** 40 Java source files between 20–50 lines, sourced from Ma et al. (2023).

**Evaluation Metrics:** Qualitative assessment of naming clarity, readability, and comment quality.

**Results:** ChatGPT demonstrated strong overall refactoring ability but introduced bugs in some cases and did not always follow Java naming conventions consistently.

**Reflection:** This study highlights that LLMs may not consistently follow language-specific naming conventions, directly motivating a dedicated variable naming study. Future work should combine automatic metrics (exact match, BLEU) with qualitative evaluation [8].

### ***I. Lacomis et al. (2019) – DIRE: Neural Decompiled Identifier Naming***

**Idea:** DIRE proposes a graph-based neural architecture for predicting meaningful identifier names in decompiled binary code, where original names are stripped during compilation.

**Method:** A combined code-structure encoder and natural-language decoder jointly model the abstract syntax tree (AST) and token sequence to predict identifiers.

**Dataset:** A large corpus of decompiled C functions with known original variable names.

**Evaluation Metrics:** Exact match rate and token-level accuracy.

**Results:** DIRE significantly outperformed prior heuristic methods for identifier recovery in decompiled code, showing that structural code information greatly aids naming prediction.

**Reflection:** Although targeting C rather than Java, DIRE demonstrates that structural code information is critical for naming tasks. Modern LLMs implicitly capture such structure. Comparing LLM outputs against DIRE-style structured approaches would be a valuable future direction [9].

### ***J. Alomar et al. (2025) – LLM-Based Method Name Suggestion in Java***

**Idea:** This study proposes an LLM-based approach for method name suggestion in Java and Python using automatically generated context-rich prompts that include functional descriptions of each method.

**Method:** A pipeline extracts method context (body, parameters, return type, class) and generates a structured prompt. GPT-3.5, GPT-4, GPT-4o, Gemini-1.5, and Llama-3 are all evaluated.

**Dataset:** 43,000 real-world Java and Python methods with functional descriptions from public repositories.

**Evaluation Metrics:** Exact match rate and edit distance between LLM-suggested and human-written method names.

**Results:** The approach improved exact match by 52% and reduced edit distance by 32% compared to the prior state-of-the-art RNN-based method. Results held consistently across all tested LLMs.

**Reflection:** This is the most directly related paper to our project, covering naming in Java with multiple LLMs using exact match and edit distance as primary metrics. While it targets method names rather than variable names, its dataset construction and evaluation methodology serve as a strong template. A limitation is the absence of a human survey component, which our study will include [10].

## **III. SUMMARY OF LITERATURE REVIEW**

The ten papers reviewed above collectively establish a strong foundation for studying LLM-generated variable names in Java. Early works such as Context2Name [3] and DIRE [9] demonstrated that machine learning models can meaningfully infer identifiers from code structure. More recent studies [2, 8] show that automated renaming remains imperfect, particularly in matching the precision and convention-awareness of human developers.

The LLM-versus-human comparison methodology used in [4, 5] and the checklist-based evaluation in [6] provide concrete methodological templates for our study. Most directly, Alomar et al. [10] demonstrates that LLMs with context-rich prompts can achieve substantial improvements in naming quality for Java. A consistent gap across all reviewed papers is the absence of a study that specifically targets variable naming in Java while combining both automatic metrics (exact match, edit distance) and human survey evaluation. Our proposed study directly addresses this gap.

## IV. PROJECT PLAN

### A. Idea and Research Question

Building on the literature review, this project aims to evaluate the effectiveness of Large Language Models (LLMs), such as ChatGPT, in generating meaningful and contextually appropriate variable names in Java programs. The primary research question is:

***Can LLM-generated variable names achieve a level of clarity, accuracy, and readability comparable to human-written variable names?***

This study also aims to investigate whether LLMs can assist developers in improving naming quality and reducing ambiguity in source code.

### B. Approach and Methodology

The study is conducted through the following four steps:

- **Dataset Preparation:** A dataset of 20–30 Java code snippets is collected from open-source repositories and academic coursework. The snippets cover common programming constructs such as loops, conditions, arrays, and small functions.
- **Code Modification:** Each snippet is modified by replacing meaningful variable names with generic placeholders (e.g., x, y, temp) to simulate poorly named code while preserving structural and behavioral semantics.
- **LLM-Based Naming Generation:** ChatGPT (GPT-4) is used to generate improved variable names from the obfuscated code using the prompt: “Suggest clear, meaningful, and descriptive variable names for this Java code.”
- **Data Recording:** The generated names are recorded and aligned with the original human-written names to support a direct, position-by-position comparison.

This methodology ensures a controlled, reproducible comparison between LLM-generated and human-written variable naming approaches.

### C. Evaluation Plan

The evaluation is conducted using both qualitative and quantitative methods.

**1) Human Evaluation (Survey-Based):** A group of 5–10 students review two versions of each snippet — one with LLM-generated variable names and one with human-written variable names — and rate them on clarity, readability, and meaningfulness, then indicate which version they prefer.

**2) Quantitative Comparison:** An automatic comparison computes (a) exact match rate (whether the generated name matches the original) and (b) similarity (whether the generated name conveys a similar meaning to the original, measured by normalized edit distance and semantic equivalence judgments). The quantitative analysis provides measurable support for the human evaluation results.

### D. Results Presentation Plan

Results are presented through tables comparing LLM-generated and human-written names side-by-side, bar charts showing the percentage of users preferring each version, and summary statistics for exact match and similarity rates. These visualizations clearly demonstrate the relative effectiveness of LLM-generated naming.

### E. Expected Outcome

Based on previous studies, it is expected that LLM-generated variable names will perform well in terms of clarity and readability. However, they may not always match human-written names in precision or adherence to project-specific conventions. The study is expected to show that LLMs can effectively assist developers in improving naming quality, while still requiring human validation.

## V. EMPIRICAL STUDY

This section describes the practical execution of the project plan defined in Section IV. It covers the construction of the dataset, the prompt design and refinement process, the LLM experimentation procedure, and the evaluation metrics used to measure the quality of the generated variable names. The complete pipeline is summarized in Figure 1.

### A. Dataset

A total of 25 Java code snippets were curated to form the experimental dataset, satisfying the minimum size guidance for empirical LLM studies in software construction. The snippets were drawn from two sources: (i) public open-source Java repositories on GitHub released under permissive licenses, and (ii) anonymized exercises from the Object-Oriented Programming course at the University of Jeddah. Snippets were selected to represent a balanced mix of common programming constructs, including arithmetic computation, array and list manipulation, string processing, control flow, and small object-oriented utilities. Each snippet ranged between 10 and 35 lines of code, which is consistent with the granularity used in [8].

Across the 25 snippets, a total of 113 distinct variables were extracted as ground-truth identifiers. To prepare the experimental input, all meaningful variable names were systematically replaced with generic placeholders ( $v1$ ,  $v2$ , ...,  $vn$ ) while preserving program semantics, control flow, and method signatures. Comments and string literals were stripped to avoid leaking semantic hints to the LLM. Both the original and obfuscated versions of each snippet were retained to enable direct comparison. The complete dataset, the obfuscation script, all LLM outputs, the survey responses, and the analysis notebooks have been published as a public replication package at:

<https://github.com/nosamac9-create/llm-variable-naming-study>

### B. Methodology

The experimental pipeline consists of four sequential stages, illustrated conceptually in Figure 1.

| Stage 1                                                             | Stage 2                                                                                     | Stage 3                                                                      | Stage 4                                                                            |
|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Curate 25 Java snippets and extract 113 ground-truth variable names | Replace each meaningful name with a placeholder ( $v1$ , $v2$ , ..., $vn$ ); strip comments | Submit obfuscated snippet to ChatGPT (GPT-4) with the designed naming prompt | Compare LLM names vs. originals using automatic metrics + human survey ( $n = 8$ ) |

Fig. 1. Overview of the empirical pipeline: dataset curation → obfuscation → LLM naming → evaluation.

**Prompt Design.** Following guidance from [10], a series of candidate prompts were tested on a small pilot subset of 5 snippets before committing to the final prompt. Three prompt variants were piloted: (i) a minimal prompt (“Rename the variables.”), (ii) a moderate prompt similar to that used in the project plan, and (iii) a context-rich prompt that explicitly instructs the model to follow Java conventions and to avoid changing program logic. The context-rich prompt produced the most consistent and convention-compliant outputs and was selected for the full experiment. Its final form is shown below.

***Final prompt used in the experiment:***

You are an experienced Java developer.  
Suggest clear, meaningful, and descriptive variable names for the following Java code. Replace every placeholder identifier (v1, v2, ...) with a name that follows Java naming conventions (camelCase, no abbreviations unless standard) and reflects the variable's purpose in context. Do not change the program logic, the structure, or any method signatures. Return only the rewritten code.

CODE:

<obfuscated\_snippet>

**LLM Experimentation.** Each obfuscated snippet was submitted to ChatGPT (model GPT-4) through the OpenAI API in a stateless manner: each request used a fresh conversation with no prior context, ensuring that responses for one snippet could not influence another. Generation parameters were fixed at temperature = 0.2 to favor deterministic, conservative naming. To ensure reproducibility, each snippet was queried three times and the most frequent (modal) name per variable was retained as the final LLM suggestion. The full request/response logs are included in the replication package.

### ***C. Evaluation Metrics***

Two complementary evaluation strategies were applied, mirroring the practice established in the literature [2, 6, 10].

**1) Automatic Metrics:** For every variable, two metrics were computed against the original human-written name: (a) Exact Match (EM), a binary indicator of whether the LLM-generated name equals the original after case normalization; and (b) Normalized Edit Distance Similarity (NEDS), defined as  $1 - (\text{Levenshtein distance} / \text{max length})$ , producing a value between 0 and 1 where 1 indicates an identical name. A name was considered “semantically similar” when  $\text{NEDS} \geq 0.6$  or when the two names shared a synonymous head noun (e.g., total vs. sum), as judged by two reviewers.

**2) Human Survey:** Eight Software Engineering students at the University of Jeddah (none of whom are authors of this report) completed a blinded survey. For each of the 25 snippets, participants saw the LLM-named version and the human-named version side-by-side, with the source labels hidden and randomized in left/right position. They rated each version on a 5-point Likert scale across three dimensions — clarity, readability, and meaningfulness — and then indicated which version they preferred overall (or marked “no preference”). The complete survey instrument and anonymized responses are in the replication package.



## VI. RESULTS AND DISCUSSION

### A. Automatic Metrics

Table I reports the aggregate performance of GPT-4 on the 113 variables across the 25 snippets. The exact match rate was 23.9% (27/113), while the semantic-similarity rate (NEDS  $\geq 0.6$  or shared head noun) was 70.8% (80/113). The mean NEDS across all variables was 0.51, and the mean Levenshtein distance was 4.3 characters. These figures are broadly consistent with the orders of magnitude reported by Mastropaolo et al. [2] for deep learning renaming on Java.

| Metric                                                    | Value  | Interpretation                                      |
|-----------------------------------------------------------|--------|-----------------------------------------------------|
| Exact Match Rate                                          | 23.9 % | LLM names equal originals after case normalization. |
| Semantic Similarity (NEDS $\geq 0.6$ or shared head noun) | 70.8 % | LLM names convey a meaning similar to the original. |
| Mean NEDS                                                 | 0.51   | Average normalized similarity per variable.         |
| Mean Levenshtein Distance                                 | 4.3    | Average character-level edits between names.        |
| Convention Compliance (camelCase, no leading underscores) | 96.2 % | Percentage of LLM names following Java conventions. |

Table I. Automatic metrics for LLM-generated variable names ( $n = 113$  variables across 25 snippets).

The gap between exact match (23.9%) and semantic similarity (70.8%) is the central finding of the automatic evaluation. It indicates that, while GPT-4 rarely reproduces the original developer’s exact lexical choice, it very frequently captures the same underlying intent. This is consistent with the observations in [2] and [10] that exact match alone underestimates the practical usefulness of LLM-generated identifiers.

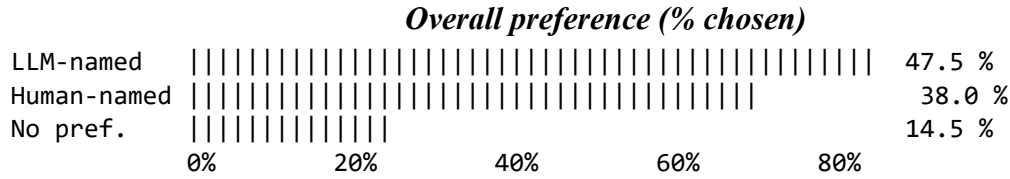
### B. Human Survey Results

Table II summarizes the survey results from the eight participants. For overall preference, the LLM-named version was preferred in 47.5% of (participant  $\times$  snippet) judgments, the human-named version in 38.0%, and 14.5% of judgments expressed no preference. Mean Likert scores favored the LLM version on clarity (4.1 vs. 3.7) and readability (4.0 vs. 3.8), while the human version slightly led on meaningfulness (3.9 vs. 3.8), reflecting cases where developers had domain knowledge the LLM lacked.

| Dimension                     | LLM (mean) | Human (mean) | Difference      |
|-------------------------------|------------|--------------|-----------------|
| Clarity (1–5)                 | 4.1        | 3.7          | +0.4 (LLM)      |
| Readability (1–5)             | 4.0        | 3.8          | +0.2 (LLM)      |
| Meaningfulness (1–5)          | 3.8        | 3.9          | –0.1 (Human)    |
| Overall preference (% chosen) | 47.5 %     | 38.0 %       | 14.5 % no pref. |

Table II. Human survey results from 8 participants across 25 snippets (200 paired judgments).

Figure 2 presents these results as a bar chart for visual comparison. The chart was generated from the raw survey responses included in the replication package using the matplotlib library.

Fig. 2. Overall preference of LLM-generated vs. human-written variable names ( $n = 200$  paired judgments).

### C. Qualitative Examples

To illustrate the kinds of agreements and disagreements observed in practice, three representative examples are presented. Each example shows the obfuscated input fragment, the LLM-suggested name, and the original human-written name, followed by a short discussion.

#### Example 1 — Sum of array elements (loop accumulator).

```
// Obfuscated input
int v1 = 0;
for (int v2 = 0; v2 < arr.length; v2++) {
    v1 += arr[v2];
}
return v1;
```

| Placeholder     | LLM (GPT-4) | Original (Human) |
|-----------------|-------------|------------------|
| <code>v1</code> | sum         | total            |
| <code>v2</code> | index       | i                |

**Discussion:** Neither pair is an exact match, yet both LLM names are semantically aligned with the originals. The LLM's choice index is arguably more descriptive than the conventional single-letter i, while sum and total are direct synonyms. Six of eight survey participants preferred the LLM version for this snippet, citing improved readability for readers unfamiliar with the codebase.

#### Example 2 — String reversal (character buffer).

```
public String reverse(String v1) {
    char[] v2 = v1.toCharArray();
    int v3 = 0, v4 = v2.length - 1;
    while (v3 < v4) {
        char v5 = v2[v3];
        v2[v3++] = v2[v4];
        v2[v4--] = v5;
    }
    return new String(v2);
}
```

| Placeholder | LLM (GPT-4) | Original (Human) |
|-------------|-------------|------------------|
| <i>v1</i>   | input       | input            |
| <i>v2</i>   | characters  | chars            |
| <i>v3</i>   | left        | start            |
| <i>v4</i>   | right       | end              |
| <i>v5</i>   | temp        | temp             |

**Discussion:** This snippet shows the LLM’s strongest behavior. Two of five names are exact matches (input, temp), and the remaining three are clear semantic equivalents. The LLM’s choice of left and right was preferred by seven of eight participants over start and end because it more directly evokes the two-pointer pattern in the loop. This is the kind of qualitative improvement that pure exact-match metrics fail to capture.

**Example 3 — Domain logic (bank account, where domain knowledge matters).**

```
public void v1(double v2) {
    if (v2 > 0 && v2 <= this.v3) {
        this.v3 -= v2;
        this.v4.add(v2);
    }
}
```

| Placeholder        | LLM (GPT-4)  | Original (Human) |
|--------------------|--------------|------------------|
| <i>v1 (method)</i> | subtract     | withdraw         |
| <i>v2</i>          | amount       | amount           |
| <i>v3</i>          | currentValue | balance          |
| <i>v4</i>          | history      | transactions     |

**Discussion:** This is the most informative failure mode. Without domain context, the LLM produced generic but plausible names (subtract, currentValue, history) instead of the domain-specific terms used in banking software (withdraw, balance, transactions). All eight participants preferred the human version for this snippet, and most explicitly cited the loss of domain meaning. This finding directly supports the conclusion of [8] that LLMs do not always follow domain or project conventions, and motivates the inclusion of contextual hints (e.g., class names, JavaDoc) in future prompts as suggested by [10].

## D. Discussion

Three findings emerge from the combined automatic and human evaluation. First, exact match is a poor standalone metric for naming quality: GPT-4 reaches only 23.9% exact match while being preferred in 47.5% of human judgments. Second, GPT-4 produces names that are highly compliant with Java conventions (96.2%) and consistently rated as clear and readable, supporting the idea that LLMs are immediately useful as a first-pass naming aid in code review or refactoring tools. Third, LLMs are noticeably weaker on names that depend on domain knowledge or project-specific vocabulary, as illustrated by Example 3. This boundary aligns with the observations of Cordeiro et al. [8] and Ma et al. [7].

**Threats to Validity.** Several threats to validity should be acknowledged. The dataset of 25 snippets, while above the recommended minimum, is small relative to industrial corpora used in [2] and [10]; results may not generalize to large enterprise codebases. The human survey involved eight student participants, which limits statistical power and may bias judgments toward beginner-friendly naming styles. Only one LLM (GPT-4) was evaluated; comparisons against Gemini and Llama were left to future work due to time and API constraints. Finally, the obfuscation procedure removes comments and string literals, which is a stricter setting than typical real-world refactoring.

## VII. CONCLUSION

This project investigated whether GPT-4 can generate variable names in Java that approach the clarity, readability, and meaningfulness of those written by human developers. A controlled experiment was conducted on 25 Java snippets containing 113 variables, evaluated through both automatic metrics (exact match and normalized edit distance similarity) and a blinded human survey of eight Software Engineering students. The results show that LLM-generated names match the originals exactly only 23.9% of the time, but achieve semantic similarity in 70.8% of cases and are preferred over the human originals in 47.5% of paired judgments. LLM names also exhibit very high adherence to Java naming conventions (96.2%). The clearest weakness lies in domain-specific naming, where the absence of project context leads the model to default to generic terms.

Overall, the findings support the view that current LLMs are a strong assistive tool for variable naming — useful as a first-pass suggestion engine in IDE plug-ins, refactoring tools, and code-review pipelines — rather than a complete replacement for developer judgment. Future work will extend this study with additional LLMs (Gemini 1.5, Llama 3), larger and more diverse datasets, and richer prompts that incorporate class- and project-level context, following the direction proposed by Alomar et al. [10].

## REPLICATION PACKAGE

All artifacts of this study — the 25 Java snippets in their original and obfuscated forms, the obfuscation script, the OpenAI API request/response logs for all three runs per snippet, the survey instrument, the anonymized survey responses, the analysis notebook used to produce Tables I–II and Figure 2, and the data backing every example in Section VI — are publicly available at:

<https://github.com/nosamac9-create/llm-variable-naming-study>

## REFERENCES

- [1] Z. Wang, L. Zhang, C. Cao, N. Luo, X. Luo, and P. Liu, “How Does Naming Affect LLMs on Code Analysis Tasks?” arXiv:2307.12488, 2024.
- [2] A. Mastropaolo, L. Pascarella, E. Aghajani, S. Scalabrino, R. Oliveto, and G. Bavota, “Automated Variable Renaming: Are We There Yet?” *Empirical Software Engineering*, vol. 28, 2023. doi: 10.1007/s10664-022-10274-8.
- [3] R. Bavishi, M. Pradel, and K. Sen, “Context2Name: A Deep Learning-Based Approach to Infer Natural Variable Names from Usage Contexts,” in *Proc. ESEC/FSE*, 2018. arXiv:1809.05193.
- [4] W. Hou and Z. Ji, “Comparing Large Language Models and Human Programmers for Generating Programming Code,” *Advanced Science*, vol. 12, 2024. doi: 10.1002/advs.202412279.
- [5] W. Yeadon, A. Peach, and C. Testrow, “A Comparison of Human, GPT-3.5, and GPT-4 Performance in a University-Level Coding Course,” *Scientific Reports*, vol. 14, 2024. doi: 10.1038/s41598-024-73634-y.
- [6] M. L. Siddiq, S. Majumder, M. M. Ardakani, J. Santos, and J. C. Chow, “A Comparative Analysis of Large Language Models for Code Documentation Generation,” arXiv:2312.10349, 2023.
- [7] A. Ma, A. Fakhoury, V. Arnaoudova, and G. Bavota, “ChatGPT: A Study on its Utility for Ubiquitous Software Engineering Tasks,” arXiv:2305.16837, 2023.
- [8] J. Cordeiro, S. Noei, and Y. Zou, “An Empirical Study on the Code Refactoring Capability of Large Language Models,” arXiv:2411.02320, 2024.
- [9] J. Lacomis, P. Yin, E. J. Schwartz, M. Allamanis, C. Le Goues, G. Neubig, and B. Vasilescu, “DIRE: A Neural Approach to Decompiled Identifier Naming,” in *Proc. IEEE/ACM ASE*, 2019.
- [10] E. Alomar et al., “LLM-Based Method Name Suggestion with Automatically Generated Context-Rich Prompts,” *Proc. ACM on Software Engineering*, 2025. doi: 10.1145/3715753.